医生排班模块讲解文档

一、数据库与实体类设计

1.1 核心实体表设计

1.1.1 排班实体表(schedule)

```sql
CREATE TABLE `schedule` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '排班ID',
  `doctor_id` bigint NOT NULL COMMENT '医生ID',
  `schedule_date` date NOT NULL COMMENT '排班日期',
  `time_slot` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NOT
NULL COMMENT '时间段(上午/下午/晚上)',
  `max_patients` int NULL DEFAULT 0 COMMENT '最大接诊人数',
  `current_patients` int NULL DEFAULT 0 COMMENT '当前预约人数',
  `status` tinyint NULL DEFAULT 1 COMMENT '状态(0:停诊,1:正常)',
  `create_time` datetime NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `update_time` datetime NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`),
  INDEX `idx_doctor_date`(`doctor_id`, `schedule_date`) USING BTREE
)
```

1.1.2 Schedule实体类

```java
@Data
@TableName("schedule")
@Schema(description = "排班实体类")
public class Schedule {
    @TableId(type = IdType.AUTO)
    @Schema(description = "排班ID")
    private Long id;

    @Schema(description = "医生ID")
    private Long doctorId;

    @Schema(description = "排班日期")
    private LocalDate scheduleDate;

    @Schema(description = "时间段(上午/下午/晚上)")
    private String timeSlot;

    @Schema(description = "最大接诊人数")
    private Integer maxPatients;

    @Schema(description = "当前预约人数")
    private Integer currentPatients;

    @Schema(description = "状态(0:停诊,1:正常)")
    private Integer status;
```

半夜撕代码版权所有

```
    @Schema(description = "创建时间")
    private LocalDateTime createTime;

    @Schema(description = "更新时间")
    private LocalDateTime updateTime;

    @TableField(exist = false)
    @Schema(description = "医生信息")
    private Doctor doctor;

    @TableField(exist = false)
    @Schema(description = "剩余可预约数")
    private Integer remainingQuota;

    public Integer getRemainingQuota() {
        if (maxPatients == null || currentPatients == null) {
            return 0;
        }
        return maxPatients - currentPatients;
    }
}
```

1.2 核心属性说明

- **id**: 排班记录的唯一标识符
- **doctorId**: 关联的医生ID，表示该排班属于哪位医生
- **scheduleDate**: 排班日期，使用LocalDate类型存储
- **timeSlot**: 时间段，可选值为"上午"、"下午"、"晚上"
- **maxPatients**: 最大接诊人数，表示该时段最多可以预约的患者数量
- **currentPatients**: 当前预约人数，表示已经预约的患者数量
- **status**: 排班状态，1表示正常，0表示停诊
- **doctor**: 非数据库字段，用于关联医生信息
- **remainingQuota**: 非数据库字段，计算剩余可预约数量

二、各功能详细讲解

2.1 排班管理功能

2.1.1 功能概述

排班管理功能允许医院管理员或医生创建、编辑、删除和查询医生的出诊排班信息。系统支持单个排班创建和批量排班创建，可以设置每个时段的最大接诊人数，并提供停诊功能。排班信息是患者预约挂号的基础数据。

2.1.2 详细讲解功能实现流程

1. **排班列表查询流程**:
   前端在vue3/src/views/backend/schedule/index.vue中，通过onMounted钩子调用fetchSchedules方法，该方法使用request.get向/schedule/page接口发送请求，传递查询参数。后端ScheduleController的getSchedulesByPage方法接收请求，调用

半夜撕代码版权所有

scheduleService.getSchedulesByPage方法进行查询，该方法使用MyBatis-Plus的LambdaQueryWrapper构建动态查询条件，最后将查询结果返回给前端。

2. **新增排班流程**：
前端点击"新增排班"按钮，弹出对话框，填写排班信息后，点击提交按钮，调用submitForm方法，该方法通过request.post向/schedule接口发送请求。后端ScheduleController的createSchedule方法接收请求，调用scheduleService.createSchedule方法，该方法首先检查医生是否存在，然后检查是否已存在相同日期和时间段的排班，最后将排班信息保存到数据库。

3. **批量排班流程**：
前端点击"批量排班"按钮，弹出对话框，选择医生、日期范围、时间段和工作日，点击提交按钮，调用submitBatchForm方法，该方法生成排班数据，然后通过request.post向/schedule/batch接口发送请求。后端ScheduleController的batchCreateSchedule方法接收请求，调用scheduleService.batchCreateSchedule方法，该方法遍历排班列表，调用createSchedule方法创建每个排班。

4. **停诊与恢复流程**：
前端点击排班列表中的"停诊"或"恢复"按钮，调用handleChangeStatus方法，该方法通过request.put向/schedule/{id}/status接口发送请求。后端ScheduleController的updateScheduleStatus方法接收请求，调用scheduleService.updateScheduleStatus方法，该方法检查排班是否存在，如果要停诊则检查是否已有预约，最后更新排班状态。

2.1.3 关键代码讲解

1. **排班创建代码**
【springboot/src/main/java/org/example/springboot/service/ScheduleService.java：createSchedule方法】

```java
@Transactional
public Schedule createSchedule(Schedule schedule) {
    // 检查医生是否存在
    Doctor doctor = doctorMapper.selectById(schedule.getDoctorId());
    if (doctor == null) {
        throw new ServiceException("医生不存在");
    }

    // 检查是否已存在相同日期和时间段的排班
    LambdaQueryWrapper<Schedule> queryWrapper = new LambdaQueryWrapper<>();
    queryWrapper.eq(Schedule::getDoctorId, schedule.getDoctorId())
                .eq(Schedule::getScheduleDate, schedule.getScheduleDate())
                .eq(Schedule::getTimeSlot, schedule.getTimeSlot());

    if (scheduleMapper.selectCount(queryWrapper) > 0) {
        throw new ServiceException("该医生在所选日期和时间段已有排班");
    }

    // 设置默认值
    if (schedule.getCurrentPatients() == null) {
        schedule.setCurrentPatients(0);
    }

    if (schedule.getStatus() == null) {
        schedule.setStatus(1); // 默认正常
    }

    // 设置创建时间和更新时间
```

```
    LocalDateTime now = LocalDateTime.now();
    schedule.setCreateTime(now);
    schedule.setUpdateTime(now);

    if (scheduleMapper.insert(schedule) <= 0) {
        throw new ServiceException("排班添加失败");
    }

    return schedule;
}
```

这段代码实现了创建排班的业务逻辑。首先检查医生是否存在，然后检查是否已存在相同日期和时间段的排班，以避免重复排班。接着设置默认值，如当前预约人数为0、状态为正常，并设置创建时间和更新时间，最后将排班信息插入数据库。

2. **批量排班生成代码**
【vue3/src/views/backend/schedule/index.vue：generateBatchSchedules方法】

```
const generateBatchSchedules = () => {
  const schedules = []
  const startDate = new Date(batchForm.dateRange[0])
  const endDate = new Date(batchForm.dateRange[1])

  // 遍历日期范围
  for (let date = new Date(startDate); date <= endDate;
date.setDate(date.getDate() + 1)) {
    const dayOfWeek = date.getDay().toString() // 获取星期几 (0-6)

    // 检查是否是选中的工作日
    if (batchForm.workDays.includes(dayOfWeek)) {
      // 为每个选中的时间段创建排班
      for (const timeSlot of batchForm.timeSlots) {
        schedules.push({
          doctorId: batchForm.doctorId,
          scheduleDate: formatDate(date),
          timeSlot: timeSlot,
          maxPatients: batchForm.maxPatients,
          currentPatients: 0,
          status: 1
        })
      }
    }
  }

  return schedules
}
```

这段代码实现了批量生成排班数据的逻辑。它遍历选定的日期范围，检查每一天是否是选中的工作日（如周一至周五），如果是，则为每个选中的时间段（如上午、下午、晚上）创建一个排班记录，最后返回生成的排班数组。

3. **排班状态更新代码**
【springboot/src/main/java/org/example/springboot/service/ScheduleService.java：updateScheduleStatus方法】

半夜撕代码版权所有

```java
@Transactional
public void updateScheduleStatus(Long id, Integer status) {
    Schedule schedule = scheduleMapper.selectById(id);
    if (schedule == null) {
        throw new ServiceException("排班不存在");
    }

    // 如果要停诊，检查是否已有预约
    if (status == 0 && schedule.getCurrentPatients() > 0) {
        throw new ServiceException("该排班已有患者预约，无法停诊");
    }

    Schedule updateSchedule = new Schedule();
    updateSchedule.setId(id);
    updateSchedule.setStatus(status);
    updateSchedule.setUpdateTime(LocalDateTime.now());

    if (scheduleMapper.updateById(updateSchedule) <= 0) {
        throw new ServiceException("排班状态更新失败");
    }
}
```

这段代码实现了更新排班状态的业务逻辑。首先检查排班是否存在，如果要将状态改为停诊（status=0），则检查是否已有患者预约，如果有则不允许停诊。最后更新排班状态和更新时间。

2.1.4 功能实现细节

1. **排班冲突检查**：系统会检查同一医生在同一日期和时间段是否已有排班，避免排班冲突。
2. **批量排班优化**：批量排班功能支持选择特定的工作日（如周一至周五）和时间段（如上午、下午、晚上），大大提高了排班效率。
3. **停诊安全检查**：当排班已有患者预约时，系统会阻止停诊操作，保护患者的预约权益。
4. **排班关联信息**：在查询排班时，系统会自动关联医生和科室信息，方便前端展示完整的排班信息。

2.2 预约挂号功能

2.2.1 功能概述

预约挂号功能允许患者根据科室和医生选择合适的排班时间进行预约。系统会显示医生的可用排班时间，包括日期、时间段和剩余名额，患者可以选择合适的时间进行预约，并填写症状描述。

2.2.2 详细讲解功能实现流程

1. **科室和医生选择流程**：
   患者首先在前端vue3/src/views/frontend/appointment/index.vue页面选择科室，系统通过fetchDepartments方法获取科室列表，患者选择科室后，系统通过fetchDoctorsByDepartment方法获取该科室的医生列表，患者选择医生后，系统设置默认日期范围为今天开始的7天。
2. **排班查询流程**：
   患者选择医生和日期范围后，系统通过fetchDoctorSchedules方法向/schedule/doctor/{doctorId}接口发送请求，传递开始日期和结束日期参数。后端ScheduleController的getDoctorSchedules方法接收请求，调用scheduleService.getSchedulesByDoctor方法，该方法查询指定医生在指定日期范围内的排班信息，并关联医生和科室信息，最后将查询结果返回给前端。

半夜撕代码版权所有

3. **预约提交流程**：
患者选择排班后，系统弹出预约确认对话框，患者填写症状描述后，点击确认预约按钮，系统通过submitAppointment方法向/appointment接口发送请求。后端AppointmentController的createAppointment方法接收请求，调用appointmentService.createAppointment方法，该方法检查患者、医生和排班是否存在，检查排班是否已满，然后创建预约记录，并调用scheduleService.updateSchedulePatients方法更新排班的当前预约人数。

2.2.3 关键代码讲解

1. **获取医生排班代码**
【springboot/src/main/java/org/example/springboot/service/ScheduleService.java：getSchedulesByDoctor方法】

```
public List<Schedule> getSchedulesByDoctor(Long doctorId, LocalDate
startDate, LocalDate endDate) {
    LambdaQueryWrapper<Schedule> queryWrapper = new LambdaQueryWrapper<>();
    queryWrapper.eq(Schedule::getDoctorId, doctorId);

    if (startDate != null) {
        queryWrapper.ge(Schedule::getScheduleDate, startDate);
    }
    if (endDate != null) {
        queryWrapper.le(Schedule::getScheduleDate, endDate);
    }

    // 只查询状态正常的排班
    queryWrapper.eq(Schedule::getStatus, 1);

    // 按日期和时间段排序
    queryWrapper.orderByAsc(Schedule::getScheduleDate)
                .orderByAsc(Schedule::getTimeSlot);

    List<Schedule> schedules = scheduleMapper.selectList(queryWrapper);

    // 查询医生信息
    Doctor doctor = doctorMapper.selectById(doctorId);
    if (doctor != null) {
        // 查询科室信息
        if (doctor.getDepartmentId() != null) {
            Department department =
departmentMapper.selectById(doctor.getDepartmentId());
            doctor.setDepartment(department);
        }
        for (Schedule schedule : schedules) {
            schedule.setDoctor(doctor);
        }
    }

    return schedules;
}
```

这段代码实现了获取医生排班列表的业务逻辑。它根据医生ID和日期范围查询排班信息，只返回状态正常的排班，并按日期和时间段排序。同时，它还查询医生和科室信息，并将其关联到排班对象中，方便前端展示完整的排班信息。

半夜撕代码版权所有

2. **前端排班展示代码**
【vue3/src/views/frontend/appointment/index.vue：排班展示部分】

```
<div v-else class="schedule-list">
  <div
    v-for="schedule in scheduleList"
    :key="schedule.id"
    :id="`schedule-${schedule.id}`"
    class="schedule-item"
  >
    <div class="schedule-date-info">
      <div class="schedule-date">{{ formatDate(schedule.scheduleDate) }}
</div>
      <div class="schedule-day">{{ getDayOfWeek(schedule.scheduleDate) }}
</div>
    </div>

    <div class="schedule-time-info">
      <el-tag :type="getTimeSlotType(schedule.timeSlot)" effect="light"
round>{{ schedule.timeSlot }}</el-tag>
      <div class="schedule-quota">剩余名额: {{ schedule.maxPatients -
schedule.currentPatients }}</div>
    </div>

    <div class="schedule-action">
      <el-button
        :type="schedule.maxPatients <= schedule.currentPatients ? 'info' :
'primary'"
        :disabled="schedule.maxPatients <= schedule.currentPatients"
        @click="handleAppointment(schedule)"
        round
      >
        {{ schedule.maxPatients <= schedule.currentPatients ? '已满' : '预约'
}}
      </el-button>
    </div>
  </div>
</div>
```

这段代码展示了前端如何渲染医生的排班列表。它遍历排班列表，显示每个排班的日期、星期几、时间段和剩余名额。如果排班已满，预约按钮会显示为"已满"并禁用。患者可以点击预约按钮进行预约。

3. **更新排班预约人数代码**
【springboot/src/main/java/org/example/springboot/service/ScheduleService.java：updateSchedulePatients方法】

```
@Transactional
public void updateSchedulePatients(Long id, int change) {
    Schedule schedule = scheduleMapper.selectById(id);
    if (schedule == null) {
        throw new ServiceException("排班不存在");
    }

    int newCount = schedule.getCurrentPatients() + change;
```

```
        // 检查是否超出最大接诊人数
        if (newCount > schedule.getMaxPatients()) {
            throw new ServiceException("超出最大接诊人数");
        }

        // 检查是否小于0
        if (newCount < 0) {
            throw new ServiceException("当前预约人数不能小于0");
        }

        Schedule updateSchedule = new Schedule();
        updateSchedule.setId(id);
        updateSchedule.setCurrentPatients(newCount);
        updateSchedule.setUpdateTime(LocalDateTime.now());

        if (scheduleMapper.updateById(updateSchedule) <= 0) {
            throw new ServiceException("排班预约人数更新失败");
        }
    }
```

这段代码实现了更新排班预约人数的业务逻辑。当患者预约或取消预约时，系统会调用此方法更新排班的当前预约人数。它首先检查排班是否存在，然后计算新的预约人数，检查是否超出最大接诊人数或小于0，最后更新排班的当前预约人数和更新时间。

2.2.4 功能实现细节

1. **排班剩余名额计算**：系统会实时计算每个排班的剩余可预约名额（maxPatients - currentPatients），并在前端展示。
2. **时间段可视化**：前端使用不同颜色的标签展示不同的时间段，如上午为绿色、下午为黄色、晚上为红色，提高用户体验。
3. **预约前置检查**：系统会在预约前检查用户是否登录、是否已完善患者信息，如果没有则引导用户完成相应操作。
4. **预约成功后处理**：预约成功后，系统会询问患者是否查看预约记录，提供便捷的导航选项。

2.2.5 与其它模块交互

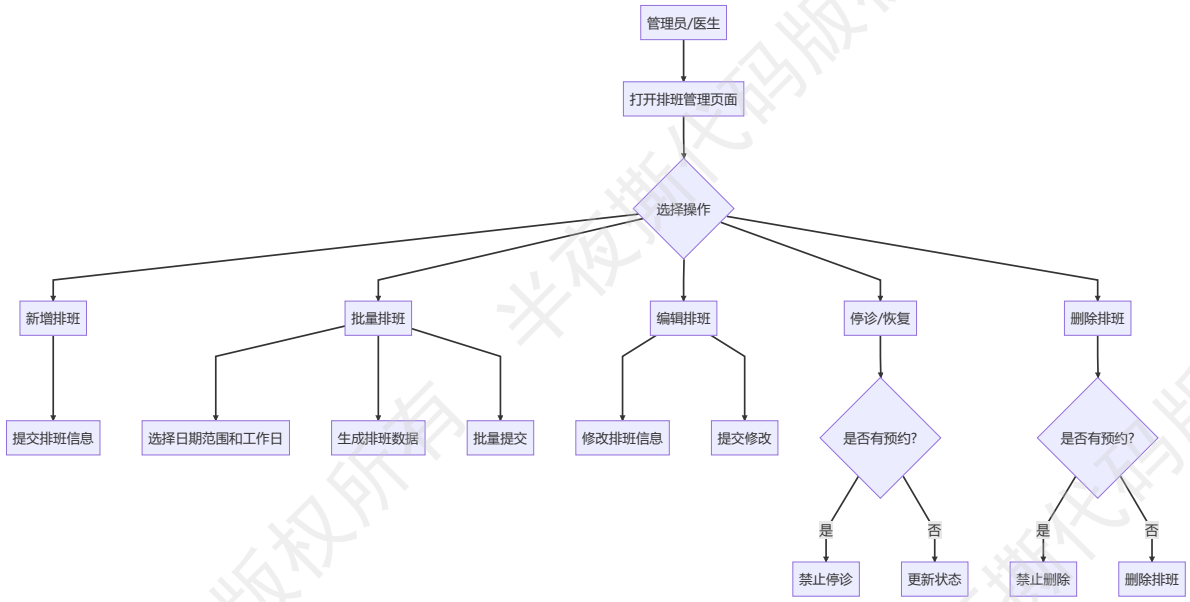
1. **与医生模块的交互**：排班模块需要获取医生信息，包括医生ID、姓名、职称等。
2. **与科室模块的交互**：排班模块需要获取科室信息，用于在排班管理和预约挂号页面展示科室名称。
3. **与预约模块的交互**：当患者提交预约时，系统会创建预约记录，并更新排班的当前预约人数。
4. **与用户模块的交互**：系统会检查用户是否登录，以及用户的角色（管理员、医生或患者）。

三、总结

3.1 排班管理功能实现

医生排班管理功能通过ScheduleController和ScheduleService实现。管理员或医生可以在后台管理页面创建、编辑、删除和查询排班信息。系统支持单个排班创建和批量排班创建，大大提高了排班效率。批量排班功能允许选择日期范围、工作日和时间段，系统会自动生成符合条件的排班记录。排班状态可以设置为正常或停诊，但当排班已有患者预约时，系统会阻止停诊操作，保护患者的预约权益。

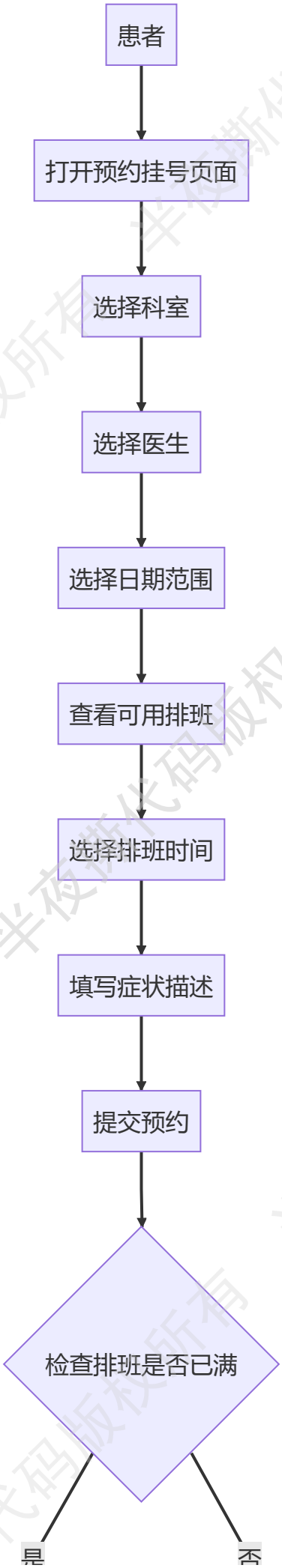
半夜撕代码版权所有



3.2 预约挂号功能实现

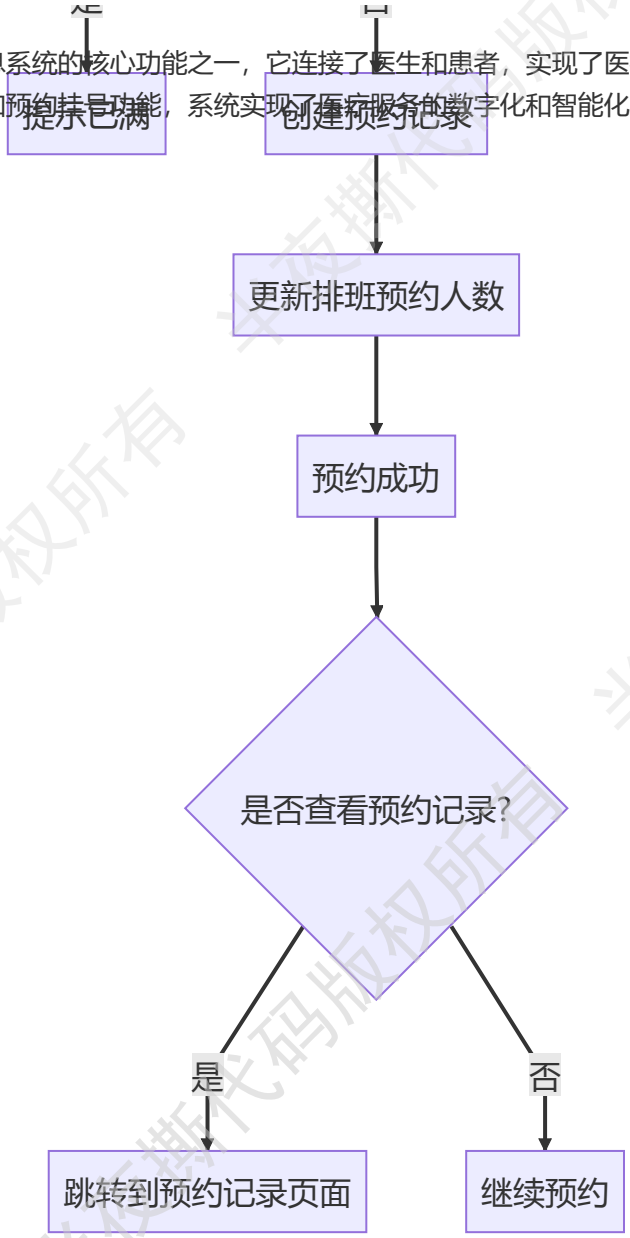
预约挂号功能通过前端appointment/index.vue页面和后端ScheduleController、AppointmentController实现。患者可以选择科室和医生，查看可用的排班时间，并进行预约。系统会显示每个排班的日期、时间段和剩余名额，方便患者选择。预约时，患者需要填写症状描述，系统会检查排班是否已满，如果未满则创建预约记录并更新排班的当前预约人数。预约成功后，系统会询问患者是否查看预约记录，提供便捷的导航选项。

半夜撕代码版权所有


患者
打开预约挂号页面
选择科室
选择医生
选择日期范围
查看可用排班
选择排班时间
填写症状描述
提交预约
检查排班是否已满
是
否

半夜撕代码版权所有

医生排班模块是医院信息系统的核心功能之一，它连接了医生和患者，实现了医疗资源的合理分配和高效利用。通过排班管理和预约挂号功能，系统实现了医疗服务的数字化和智能化，提高了医院的管理效率和患者的就医体验。



半夜撕代码版权所有